

AutoTraQ's frontend is the part of the app users actually see and interact with in the browser. It's a **single-page application (SPA)** built using **React**, which means the app feels fast because it loads once and then switches "pages" without reloading the whole site. The project uses **Vite** as the development server and build tool, which makes the app start and refresh quickly during development. It also uses **TypeScript** to help prevent bugs by catching type errors while coding, and **Tailwind CSS** for styling by using pre-made utility class names directly in the JSX instead of writing lots of separate CSS files.

The frontend code is organized so it's easy to find things. The app starts in `main.tsx`, and `App.tsx` is where the main layout and all the routes (page paths) are defined using **React Router**. The `api/client.ts` file is important because it centralizes all communication with the backend—whenever the frontend needs parts, vehicles, inventory events, or anything else, it makes those REST API calls through that one client file. This keeps the code cleaner and easier to update because you don't have API calls scattered everywhere.

The UI itself is built out of reusable components in the `components/` folder. For example, `Layout.tsx` controls the overall layout like the sidebar and header, `ProtectedRoute.tsx` blocks pages that require a logged-in user, and features like the command palette (`CommandBar.tsx`), barcode display (`Barcode.tsx`), part image viewing (`ImageGallery.tsx`), category browsing (`HierarchyBrowser.tsx`), and notifications (`NotificationBell.tsx`) are all separated into their own components so they can be reused across different pages. The `pages/` folder contains the actual screens users visit, like the Dashboard, Parts, Inventory, Vehicles, Requests, CSV import/export, scanning, admin user management, and the audit trail viewer.

Login and permissions are handled through `AuthContext.tsx`. When a user logs in, they receive a **JWT token**, and that token (plus the user info and role) is stored in the auth context so the rest of the app can access it. Pages that require authentication are wrapped in `ProtectedRoute`, which checks whether the user is logged in (and can also check their role). Styling is done with Tailwind classes right inside components, and dark mode is handled using `ThemeContext.tsx`, which toggles the theme across the app.

To run the frontend locally, you install dependencies inside the frontend folder and start the dev server (usually `http://localhost:5173`). The frontend depends on the backend, so the backend should be running first (typically on port 3002), otherwise API calls won't work. For meeting talking points, the main highlights are that it's a React SPA built with TypeScript and Tailwind for speed and reliability, Vite makes development much faster than older bundlers like Webpack, the API client keeps backend communication organized, auth context + protected routes enforce JWT login and role-based access, the dashboard uses Recharts for charts, skeleton loading states make the UI feel polished while loading, and the command bar gives quick search/navigation across the app.